TIKO — Architecture Document

Oct 10, 2026 · @ki

1. Overview

TIKO is a local-first progressive web app: every screen reads and writes a database on the device, a shared rules engine runs on both device and server, and the server exists to back up data, link devices and send reminders. This makes logging fast and offline-proof, which the product depends on (a log in under 30 seconds a day, no lost entries).

This document describes how the system is put together and why. Field-level detail is in the Database Design and API Documentation; build steps and tests are in the Technical Specification. Stack choices are proposals because the Tech Stack document was not available; where it differs, it wins.

What drives the design

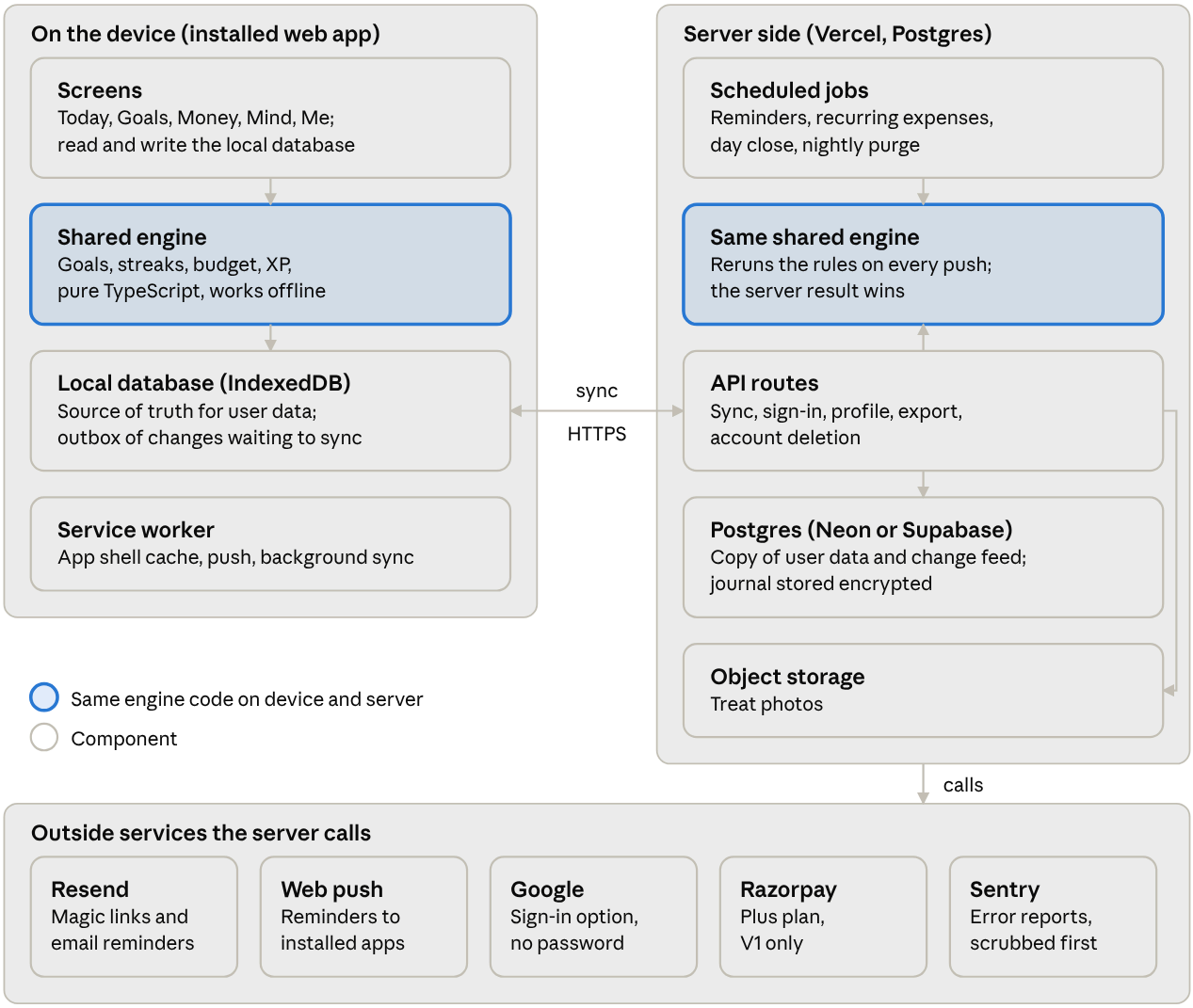
Driver	Source	Architectural answer
A tap shows in 100 ms and saves in 300 ms	NFR-02	Write to IndexedDB first; the network is never in the tap path
No log lost offline, on tab close or on failed sync	NFR-04, FR-SYN-06	Outbox, tombstones, no destructive sync
Works with no connection after first visit	NFR-11	Service worker caches the app shell; local data
Goal celebration the instant a target is met, even offline	FR-GOL-07, FR-SYN-05	Goal engine runs on the device
Journal and mood are sensitive	NFR-06, FR-MND-05	Encrypted journal, minimal analytics, no ads
One developer, free-tier infrastructure	PRD constraints	One codebase, serverless, few moving parts
Money and streak math must never disagree between devices	FR-ENG-01, FR-MON-06	One pure engine package used on both sides

Principles

- The device is the source of truth for the user's own data; the server is a copy.
- Progress, streaks and budgets are derived from logs, not stored as running totals.
- One engine, two runtimes: the same TypeScript runs in the browser and on the server.
- Fail soft: a down server, push service or email provider never blocks logging.
- Collect little: store only what a feature needs; keep text and amounts out of analytics.

2. System at a glance

The device holds the data and the rules; the server backs up and reminds



system at a glance · device, server, data, outside services

Every screen talks only to the local database and the shared engine on the device. Sync is the one link to the server, which stores a copy, reruns the same rules, sends reminders through outside services and never sits in the path of a tap.

3. Client architecture

The client is a Next.js (React, TypeScript) app installed from the browser. It is built in four layers, and each layer may call only the one below it, so a screen can never reach the network directly.

Layer	Holds	Rule
Screens	Today, Goals, Money, Mind, Me, onboarding, celebration, breathing view	Render from live queries; send user actions to the domain layer; no fetch calls
Domain	Actions such as log a habit, add an expense, claim a treat, plus the shared engine	Runs the engine, writes the result in one local transaction, queues a change for sync
Local store	IndexedDB through Dexie: one store per synced entity, the outbox, settings, sync cursor	The source of truth; live queries push changes back to screens
Sync and platform	Sync client, service worker, push registration, notification permission, install prompt	The only code that touches the network or browser platform APIs

How the main pieces behave

- **Writes.** A tap becomes one Dexie transaction holding the log, any recomputed goal rows and an outbox entry. If it commits, the card updates and the entry is safe (NFR-02, NFR-04). The 10-second undo reverses that transaction's rows rather than deleting data.
- **Reads.** Screens subscribe to live queries, so a change from sync, another tab or the engine appears with no manual refresh. Today renders from local data and the cached shell, not from a server response (NFR-01).
- **UI state.** A small store (Zustand) holds only transient state such as open sheets and undo timers. There is no server-state cache because the screens do not fetch.

- **Service worker.** It precaches the app shell and fonts so the app opens offline after the first visit (NFR-11), receives web push, and triggers background sync where the browser allows it. Browsers without Background Sync rely on the sync client running at app open, on focus and on a timer.
- **Sync client.** It sends the outbox in batches, applies pulled changes to the local store, retries with backoff capped at 30 seconds and shows "Saved on this device" while the outbox has entries. It never deletes a local row because a request failed.
- **Timers.** A timer habit stores its start time and computes elapsed time on demand, so closing the tab loses nothing.
- **Guests.** Guest mode uses the same local store with no account; first sign-in uploads the whole dataset once.

Client modules (one repository, one package boundary each)

Module	Responsibility
`engine`	Pure functions for goals, streaks, budget, XP, low-mood rules; no I/O; shared with the server
`db`	Dexie schema, migrations, entity types, outbox
`sync`	Push, pull, merge-guest, conflict handling
`features/*`	One folder per area: habits, goals, money, mind, me
`ui`	Design-token components, theme, accessibility helpers
`platform`	Service worker, push, install, storage persistence

Where the client is weak. Browsers may clear site data for guests who never install or return, Safari most aggressively. The client asks for persistent storage, prompts installation, offers sign-in after the first completed goal and keeps export available, but it cannot fully remove this risk.

4. Server architecture

The server is a set of stateless Next.js route handlers on Vercel, plus scheduled jobs, in front of one Postgres database. Nothing is kept in server memory between requests, so any request can land on any instance.

Component	Job	Notes
Route handlers	Sync (push, pull, merge-guest), profile, notification settings, push subscriptions, export, account deletion, photo upload	Validate with Zod; scope every query by the session's user; see the API Documentation
Auth	Magic link by email and Google sign-in; session cookies	Auth.js, or Supabase Auth if Supabase is chosen; one of the two, not both
Sync service	Applies each mutation once, resolves conflicts, assigns the change sequence, returns recomputed rows	Idempotent by `op_id`; latest `updated_at` wins per row
Shared engine	Recomputes goals, streaks and XP after each push	The same package the client runs; the server result is authoritative
Crypto service	Encrypts and decrypts journal bodies	AES-256-GCM with a per-user key wrapped by a master key held outside the database
Notification service	Chooses push or email, applies the 2-a-day and quiet-hour rules, picks tone copy	Removes dead push subscriptions and falls back to email
Scheduled jobs	Reminders, close of ended days, recurring expenses, nightly purge, weekly recap (V1)	Called by an external scheduler with a secret; each job is safe to run twice
Billing (V1)	Razorpay orders and signed webhooks	Not in the MVP

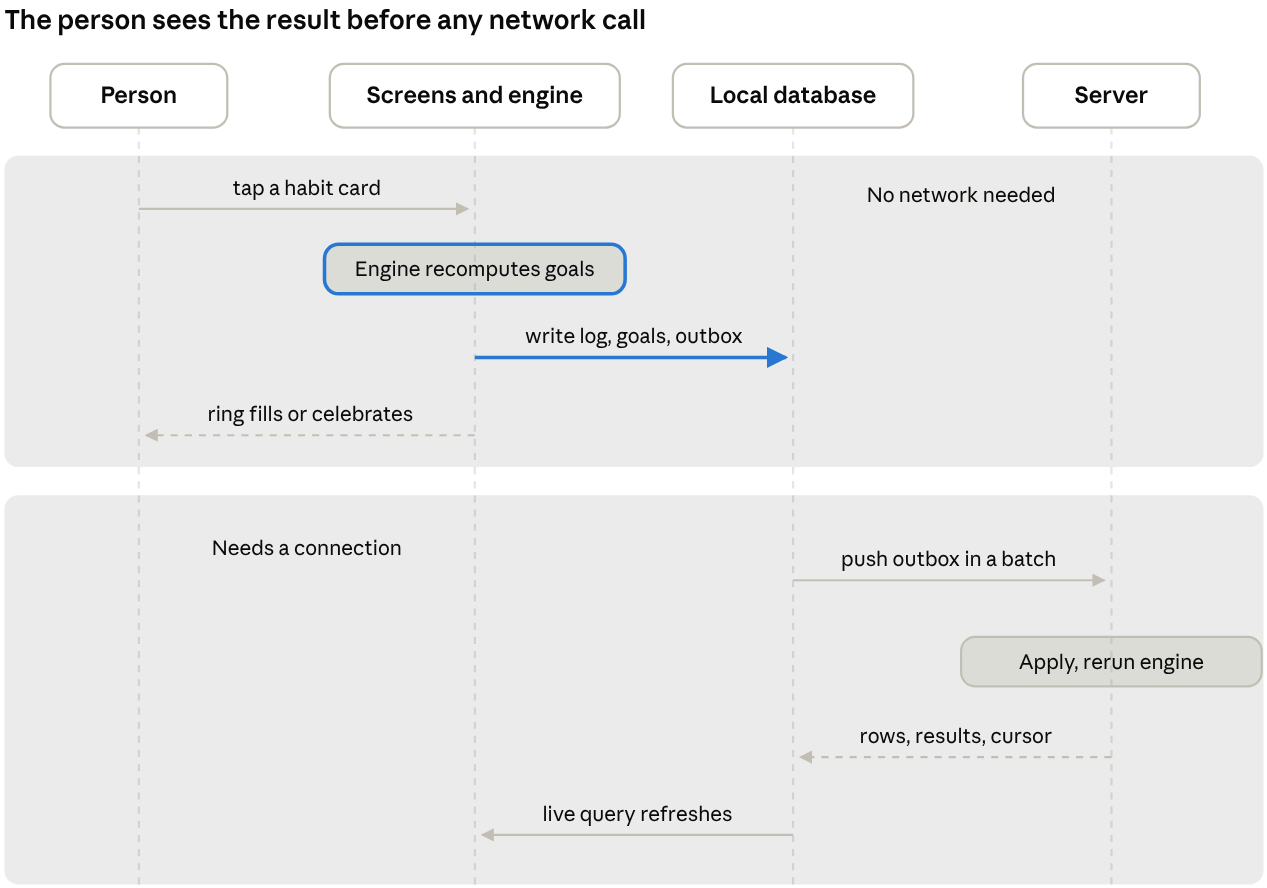
Rules every request follows

1. Check the session and the request origin; reject otherwise.
2. Validate the body against its schema; reject unknown fields.
3. Check rate limits (kept in a small Redis-style store, not in Postgres).
4. Run the work in one database transaction, with the user id set for row-level security.
5. Return per-mutation results for sync, or the standard error shape for everything else.

Why a monolith. One developer, one deploy, one language and one engine package means no service boundaries to version or keep in sync. The only seams are the engine package, which is shared by design, and the scheduler, which is external because reminder precision needs about 5-minute runs.

Failure behavior. If the database is down, sync routes return 503 and devices keep working and queue changes. If the push service or email provider fails, the notification service retries and falls back to the other channel. If the scheduler misses a run, the next run catches up because jobs look at what is due, not at when they were called.

5. Key flows



log a habit and complete a goal · device first, sync later

Everything above the second band happens on the device in one local transaction. Sync runs later, and the same steps work offline, only waiting for a connection before the push.

What happens at the edges

- **Goal completes offline.** The device engine detects completion in the same transaction and shows the celebration at once. When the server reruns the engine it confirms the goal; if it disagrees, because another device changed a log, the client adopts the server's state quietly. A claimed treat is never taken back and no failure wording is shown.
- **Same entry edited on two devices.** The later `updated_at` wins for that row; a stale write comes back as `stale` and the client adopts the server row.
- **Retry after a lost response.** Each mutation carries an `op_id`, so a resent batch is recognized and not applied twice.
- **First sign-in after guest use.** The whole local dataset is uploaded once; the server matches core habits by key, rewrites child rows to the account's ids and returns an id map the client applies.
- **Reminder.** A scheduled job finds users whose reminder time has come, checks the 2-a-day and quiet-hour limits, and sends push or email. The service worker shows the push; tapping it opens Today.

6. Deployment and environments

One repository deploys to Vercel; the database and scheduler are separate managed services, so any of them can be swapped without touching the others. The PRD lists the hosting and database providers as dependencies with a fallback for each: move to another provider, with data export built in.

Environment	Purpose	Database	Notes
Local	Development	Local Postgres in a container	Seed script, test accounts; email and push stubbed
Preview	One per pull request	A branch or throwaway database	Runs end-to-end and Lighthouse checks
Staging	Beta testers and release checks	Own database, same version as production	Used for the beta gate and security scan
Production	Real users	Production database with backups of 30 days or less	Deploys only from the main branch after checks pass

Delivery pipeline. Every pull request runs typecheck, lint, unit and property tests, integration tests against a real Postgres, Playwright flows, an accessibility scan and a Lighthouse performance budget. Merging to main deploys to staging; promoting to production is a manual step until the beta gate is met. Database migrations run before the new code starts, written so the previous release still works (add, backfill, then remove).

Configuration and secrets. Secrets (session secret, email key, push keys, master encryption key, scheduler and webhook secrets) live only in the hosting provider's environment settings, one set per environment, never in the repository. The master key is kept apart from the database so a database leak does not expose journal text.

Operations

- **Monitoring:** Sentry for errors with personal content scrubbed; an uptime monitor against a health route for the 99.5% a month target (NFR-03, still to validate).
- **Backups:** daily database backups kept 30 days at most, so deleted accounts disappear within the 30-day promise (NFR-07). Restore is tested before public launch.
- **Scaling:** serverless functions scale with traffic; the database is the limit. One Postgres instance is planned to 50,000 users (NFR-12); free-tier storage will not last that far.
- **Rollback:** redeploy the previous build; because migrations are backward compatible, no data rollback is needed. Cached clients are told to reload if their schema version falls below the supported minimum.

7. Security and privacy architecture

There are three places data lives: the device, the server and a small set of outside services. Each has a different trust level, and the rules below are what the architecture enforces at each boundary.

Data classes and where they may go

Data	Examples	Device	Server database	Outside services and analytics
Mind data	Journal bodies, mood notes, tags	Yes, unencrypted in local storage	Journal body encrypted; mood stored as entered	Never; not in analytics, push or email

Data	Examples	Device	Server database	Outside services and analytics
Money data	Expense amounts, budgets	Yes	Yes	Amounts never in analytics, push or email
Habit and goal data	Logs, goals, treats	Yes	Yes	Event names only, no content
Identity	Email, name, time zone, currency	Yes	Yes	Email to Resend for magic links and reminders
Credentials and keys	Magic-link tokens, session, encryption keys	Session cookie only	Tokens hashed; user keys wrapped by a master key held outside the database	Never
Treat photos	Optional image	Yes	In object storage, linked by key	Never

Trust boundaries and controls

1. **Browser to server.** HTTPS only with HSTS, a strict Content Security Policy, SameSite cookies plus an origin check on writes, and input validation on every route (NFR-05).
2. **Request to data.** Every query is scoped by the signed-in user; row-level security in Postgres is a second lock, set per transaction, so a missed filter cannot expose another person's rows.
3. **Application to database.** The app role cannot alter the schema; a separate role owns migrations. Reporting or analytics roles cannot read journal, mood or expense tables.
4. **Server to outside services.** Only the minimum leaves: an email address and notification copy to Resend, an anonymous push payload to push services, scrubbed errors to Sentry. Notification text never carries journal text or exact amounts (FR-NOT-06).
5. **Journal at rest.** AES-256-GCM per entry with a per-user key, wrapped by a master key kept in the hosting provider's secret store or a KMS. This is encryption at rest, not end-to-end; the operator could decrypt. End-to-end would break magic-link account

recovery and server-side export, and is not planned.

Privacy behaviors the architecture must support

- **Export** of everything the user owns, mind data included, as CSV and JSON (FR-SET-02).
- **Deletion:** access ends at once; rows are purged by a nightly job within the 30-day limit (targeted within 7 days); backups age out within 30 days (NFR-07).
- **No ads, no sale of data, no leaderboards** (FR-ENG-08); mind data is never used for ads or shared (FR-MND-05).
- **Self-harm wording** in a journal is detected on the device while the user types, against a maintained list, so no text is sent to the server for this purpose. The support card appears calmly and never locks the user out (FR-MND-09).

Open risks. The local copy of the journal on the device is not encrypted by the app, so the help page should say so. DPDP Act 2023 and GDPR review, with legal advice, is a release gate before public launch (NFR-16); how mood data is classified under them is for that review to decide.

8. How the architecture meets the NFRs

Each target in the PRD maps to a structural decision, and each decision has a cost the team accepts. The per-NFR tests are in the Technical Specification.

Quality	NFR	Decision	Cost accepted
Speed	NFR-01, 02	Local-first writes and reads; cached shell; code splitting	More client code and a local database to migrate
Reliability	NFR-03, 04	Server is not in the tap path; outbox with retries; tombstones	Two copies of data to reconcile
Offline	NFR-11, FR-SYN-*	Service worker plus IndexedDB; sync later	Browser storage can be evicted for guests; Safari is the weakest
Consistency	FR-ENG-01, FR-MON-06	One pure engine package on both sides; server result wins	Client and server must ship compatible engine versions

Quality	NFR	Decision	Cost accepted
Conflict handling	FR-SYN-04	Latest edit wins per row	A simultaneous edit on two devices loses the earlier one
Security	NFR-05	HTTPS, cookies, validation, rate limits, row-level security	Extra setup per request; a second lock to maintain
Privacy	NFR-06, 07	Encrypted journal, analytics allowlist, deletion job, short backups	Server-side encryption only; no end-to-end privacy from the operator
Accessibility	NFR-08, 09	Design tokens carry contrast; shared components carry focus and labels	Component discipline; no one-off controls
Compatibility	NFR-10	Latest two versions of four browsers, from 360 px	iOS web push only for installed apps; email fills the gap
Scalability	NFR-12	Stateless functions; one Postgres with targeted indexes; paged sync	Database is a single point; redesign beyond about 50,000 users
Quality	NFR-13	Engine isolated as pure code so tests are cheap and complete	Strict module boundaries
Observability	NFR-14	Sentry with scrubbing; allowlisted events	Less debugging detail than full logging
Localisation	NFR-15	Message files and Intl for money and dates	Discipline in not hardcoding strings
Cost and effort	PRD constraints	Monolith, serverless, managed services	Vendor limits on free tiers; scheduler precision depends on the plan

Where the design is under most strain

- **Clock skew.** Latest-edit-wins uses client time, which can be wrong; the server clamps timestamps more than 5 minutes ahead. Because most data is append-only logs, real conflicts are rare.
- **Out-of-order change numbers.** A change sequence number can commit after a later one; the client re-reads a small overlap on each pull to avoid skipping rows.
- **Reminder timing.** Reminders need runs about every 5 minutes, which a free hosting plan may not provide; an external scheduler is the assumed answer.
- **Mind data outside the journal.** Mood notes and tags are stored unencrypted on the server. Only the journal body is required to be encrypted (NFR-06), so this is a conscious boundary that the privacy review should confirm.

9. Decisions and evolution

Architecture decisions. Status shows what is settled by the documents and what is proposed here until someone confirms it.

#	Decision	Why	Status
1	Progressive web app first, no native apps in the MVP	PRD scope; fastest path for one developer	Settled (PRD)
2	Local-first with a server copy	Speed and offline are product promises	Proposed
3	One Next.js codebase for UI and API	PRD simplified single-codebase direction	Settled (PRD)
4	Pure shared engine package	Same rules on device and server; cheap tests	Proposed
5	Progress derived from logs, never incremented	Undo, edits and merges stay correct	Proposed
6	Latest edit wins per row, with timestamp clamping	FR-SYN-04	Settled (FR), clamp proposed
7	Neon with Auth.js, or Supabase for both	One provider choice for database and sign-in	Open
8	External scheduler for 5-minute jobs	Hosting plan limits on cron frequency	Open

#	Decision	Why	Status
9	Journal encrypted at rest on the server, not end-to-end	Search, export and account recovery	Proposed; confirm
10	Self-harm wording detected on the device	No journal text sent for detection	Proposed
11	Money as integer paise; TIKO never moves money	FR-MON-03, FR-MON-13	Settled (FR)

Evolution

Release	Architectural change	What stays the same
MVP (weeks 1 to 8)	The architecture above: PWA, local-first, sync, engine, scheduler	n/a
V1 (weeks 9 to 16)	Savings goals and contributions, weekly recap job, health-app step import where the platform allows it, Plus plan with Razorpay billing and entitlement checks	Local-first model, sync routes, engine package; new entities are added to sync, not a new path
V2 (weeks 17 to 26)	A native wrapper that reads the pedometer and passes store review; more languages; coach-led programmes	The same web app inside the wrapper; the same API and data model

Why this evolves cleanly. New features add an entity to the sync list, a rule to the engine and a screen, rather than a new communication path. Entitlements are data, so billing changes limits without code changes. The wrapper reuses the web app, so the PWA's offline and sync work is not thrown away.

Gates between releases (from the PRD). MVP to V1 needs 10 beta users logging 4 or more days a week; V1 to V2 needs day-30 retention near 20%. Architecture review should happen at each gate: database size and cost, scheduler precision, sync conflict rates and engine test coverage are the figures to check.